EPIC-G — Notifications (In-App and Email) — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The seventh per-epic technical spec. Builds on the architecture spec's Section 4.2 (`community.notifications`, `community.notification_preferences`), Section 4.1 (`NotificationService` as one of only three modules with an `identity`-schema grant), and Section 7.3 (notification types and delivery mechanism) — read those first.

Source of truth: `docs/askapeer-prd-v0.1.md`, the Must-have "Notifications" row (Section 6.1) and the Should-have "Email digest" row.

Contents

---

1. Scope

In scope: the in-app notification store, delivery preferences, email delivery via `NotificationService`, and the Should-have weekly digest.

Out of scope: the domain events themselves (new comment, kudos awarded, verification decision) — those are EPIC-C, EPIC-D, and EPIC-A/EPIC-F's data; this epic only reacts to them.

---

2. Data model

Reuses `community.notifications` and `community.notification_preferences`, already defined in the architecture spec, Section 4.2:

```
community.notifications
  id, handle_id, type, payload jsonb, read_at, created_at

community.notification_preferences
  handle_id, type, in_app_enabled, email_enabled
```

No new tables are needed for the in-app/email mechanics themselves; this spec's additions are behavioral (Sections 4, 6, 7), not schema.

---

3. `NotificationService`'s dual-schema access

The architecture spec (Section 4.1) grants `NotificationService` — alone among community-facing services, alongside `IdentityService` and `BillingService` — a database role with access to the `identity` schema, specifically to read `identity.members.email` for sending mail. This is worth being precise about in this spec, since it's the one place this epic touches the identity boundary at all:

`NotificationService` **should read only** `email` **from** `identity.members`**, never** `legal_name`**.** Nothing in the architecture spec technically prevents a wider read, since the grant is at the schema/table level, not column level — but every email template this service sends must address the member by their **handle**, never their real name, even though the service technically has the database access to do otherwise. This is an application-level discipline point worth stating explicitly rather than assuming it falls out of the schema separation automatically, since — unlike the general `identity`/`community` boundary, which the architecture spec enforces structurally (Section 4, via role grants) — this specific narrower constraint (email-yes, name-no) is *within* a single already-granted role, and structural enforcement (e.g., a view exposing only `email`) is a cheap and worthwhile hardening step to actually build, not just document as a norm. Flagged in Section 11.

This routine access is not an `identity_access_log` event, per the architecture spec's Section 4.4 distinction (routine automated access vs. moderator-initiated identity lookup) — consistent with what that section already states about sending a notification email.

---

4. The pre-handle notification gap

`community.notifications` **is keyed by** `handle_id` — but `verification_status_change` (one of the five notification types the architecture spec names in Section 7.3) needs to reach applicants in `pending`, `needs_more_info`, or `rejected` status, none of whom have a handle yet (per the EPIC-A/EPIC-B handoff: a handle is only created once `approved_verified` is reached). A rejected applicant, by definition, **never** gets a handle at all.

This means `verification_status_change` notifications for a not-yet-verified applicant cannot be stored in `community.notifications` — there's no `handle_id` to attach them to. This spec proposes: for these pre-handle cases, the notification is **email-only**, sent directly by `NotificationService` reading `identity.members.email`, with no `community.notifications` row created at all. Once a member reaches `approved_verified` and has a

handle, subsequent status-relevant events (e.g. a later suspension) can use the normal handle-scoped in-app-plus-email path like any other notification type.

This is a genuine structural asymmetry among the five notification types worth surfacing clearly rather than quietly special-casing in code with no documentation trail — `verification_status_change` is really two different mechanisms depending on whether a handle exists yet, not one mechanism with a shared table.

---

5. Notification types and triggers

Type	Trigger	Owning epic (event source)	Handle exists?
`reply`	New comment on a handle's post/comment	EPIC-C	Always
`mention`	`@handle` parsed in a post/comment body	EPIC-C	Always
`kudos_received`	Kudos awarded to a handle's post/comment	EPIC-D	Always
`verification_status_change`	Any `identity.verification_decisions` transition	EPIC-A (and EPIC-F, for suspend/expel)	Sometimes — Section 4
`weekly_digest` (Should-have)	Scheduled job	EPIC-C/EPIC-D data	Always

Each of the first three is delivered via a BullMQ worker job reacting to the triggering domain event, per the architecture spec's general worker pattern (Section 3) — this epic doesn't introduce a new delivery mechanism, only the specific job definitions.

---

6. Preferences — and what can't be disabled

`community.notification_preferences` lets a member configure `in_app_enabled`/`email_enabled` **per type** — but this spec proposes that `verification_status_change` is not fully configurable: a member cannot disable the **email** channel for this type, even though they could disable email for `reply` or `kudos_received`. Account-status events (verification decisions, and — once EPIC-F exists — suspension/expulsion notices) are not engagement content a member should be able to silently opt out of; a member needs to know their access has changed regardless of their notification preferences. In-app is moot for the pre-handle cases (Section 4), and for post-handle status changes there's no strong reason to force in-app specifically, only to guarantee email delivery. Flagged in Section 11 as a proposal, not a confirmed policy.

---

7. Weekly digest, built on EPIC-B's unified follows table

Resolved 2026-07-14 — this section originally flagged a gap: the PRD's Should-have digest (Section 6.1, "top-kudos content in **followed tags**") and EPIC-C's Should-have personalised feed (Section 6.1, "tags **and handles** followed") both assumed a tag-follow mechanism that no spec had built, since EPIC-B originally only specified handle-follows. Adrian's decision (see `docs/2026-07-14-technical-specs-open-questions.md`, Section 2): rather than add a separate tag-follow table, EPIC-B's `community.handle_follows` was generalised into `community.follows` (`target_type` enum of `handle`/`tag`, same discriminator pattern as `community.kudos`/`community.reports`) — see EPIC-B's spec, Section 8.

This epic builds the digest as a scheduled BullMQ job querying each member's `community.follows` rows (both `target_type = tag` and `target_type = handle`) joined against the prior week's kudos-ranked content, delivered by email only (a weekly digest has no obvious in-app equivalent). This epic is a read-only consumer of `community.follows`, same as EPIC-C's personalised feed (its spec, Section 8) — neither epic owns or duplicates the table.

---

8. API endpoints

```
GET   /v1/notifications?cursor=...&unread_only=
PATCH /v1/notifications/:id/read
POST  /v1/notifications/read-all

GET   /v1/notification-preferences
PUT   /v1/notification-preferences        { type, in_app_enabled,
email_enabled }
```

`PUT .../notification-preferences` rejects an attempt to set `email_enabled = false` for `verification_status_change`, per Section 6, if that proposal is confirmed.

---

9. Non-functional notes specific to EPIC-G

- **Email deliverability**: sent via SES in `eu-west-1` per the architecture spec, Section 6 — no new infrastructure needed here.
- **Unsubscribe/opt-out compliance**: under UK PECR, marketing-flavoured communications generally require an easy opt-out; the weekly digest plausibly falls into that category (it's engagement content, not a transactional account notice) and should carry an unsubscribe link, whereas `verification_status_change` and other account-critical notices are transactional and not subject to the same requirement — consistent with why Section 6 proposes making the latter non-optional. Worth a specific legal sanity-check alongside the DPIA already flagged in the architecture spec (Section 11) rather than assumed correct by this spec's own reasoning.
- **Payload shape**: `community.notifications.payload` is `jsonb` and type-specific (a `reply` notification's payload differs from a `kudos_received`

one) — this spec doesn't fix a schema per type here since it's an implementation detail with no architectural weight, but each type's payload shape should be documented wherever the notification-rendering UI is actually built.

---

10. Test plan

- **Pre-handle delivery**: a `rejected` applicant (no handle) receives an email notification and no `community.notifications` row is ever created for them.
- **Non-optional email**: an attempt to disable `email_enabled` for `verification_status_change` is rejected (pending Section 6's confirmation).
- `NotificationService` **field access**: an automated test asserting the service's queries against `identity.members` never select `legal_name`, only `email` — mirrors the access-boundary tests the EPIC-A and EPIC-B specs already establish for their own modules.
- **Mention parsing**: mentioning an `expelled` handle doesn't error or leak status, consistent with EPIC-C's Section 9 note on the same behavior.

---

11. Open questions

- **Column-level hardening of** `NotificationService`**'s identity access** (Section 3): should this be enforced by a database view exposing only `email`, rather than relying on application discipline over a full-table grant? Recommended, not yet built.
- **Non-optional** `verification_status_change` **email** (Section 6): needs confirmation — is it acceptable to remove member control over this one notification channel?
- ~~**Missing tag-follow mechanism**~~ — **resolved 2026-07-14**, see Section 7 above and EPIC-B's spec, Section 8.
- **Digest cadence/unsubscribe mechanics**: this spec assumes weekly, email-only, per the PRD's own naming — no further design beyond that assumption has been done here.